CSU Vehicle Speed Tracker

Real-Time Vehicle Speed Estimation from a Webcam Feed Using Classical OpenCV

CSC 126 Computer Vision – Midterm Project | Caraga State University, College of Computing and Information Sciences

Members: [Name 1], [Name 2], [Name 3] | Date: October 2026

1. Objectives

- Build a Python/OpenCV application that detects vehicles in a live webcam feed inside the CSU campus and estimates the speed of each one in km/h.
- Combine at least three laboratory topics: image operations, image segmentation, image analysis and transformation, and motion and object tracking.
- Display processed frames with bounding boxes, IDs, speeds, a motion mask and a vehicle counter, and flag vehicles that exceed the campus speed limit.

2. Real-World Problem

Campus roads are shared by students, pedestrians, motorcycles and cars. Speeding near classrooms and crossings is a safety risk, but permanent speed radars are expensive and security staff cannot watch every road at all times. A single fixed camera combined with classical computer vision can measure vehicle speeds, count traffic, and highlight vehicles over the limit at very low cost, using only a laptop and a webcam.

3. Techniques Used

Topic	OpenCV functions	Role in the project
a. Image Operations	resize, cvtColor, GaussianBlur, morphologyEx, dilate	Frames are resized, converted to grayscale and blurred to reduce noise. Opening, closing and dilation clean the motion mask (remove specks, fill holes, merge vehicle parts).
b. Image Segmentation	createBackgroundSubtractorMOG2, threshold	MOG2 learns the static road as background; moving pixels become the foreground. Thresholding at 200 removes the shadow pixels (value 127).
d. Image Analysis and Transformation	findContours, contourArea, boundingRect, pointPolygonTest, getPerspectiveTransform, perspectiveTransform	Contours of the mask are filtered by area and by location inside the calibrated road. A perspective transform (homography) maps image pixels to real meters on the road plane.
e. Motion and Object Tracking	centroid tracker (tracker.py)	Each vehicle gets a unique ID by nearest-centroid matching between frames. Its world positions over time give its speed.

Speed estimation. The user clicks four points forming a rectangle on the road and enters its real width and length in meters. The homography converts each vehicle centroid to meter coordinates (X, Y). Speed is the distance between the oldest and newest points in a short window of the vehicle's history divided by the elapsed time, multiplied by 3.6 to get km/h. The median of the last 10 readings is displayed to suppress jitter. Timestamps come from the real clock for a webcam and from frame number divided by FPS for video files.

4. Results and Screenshots

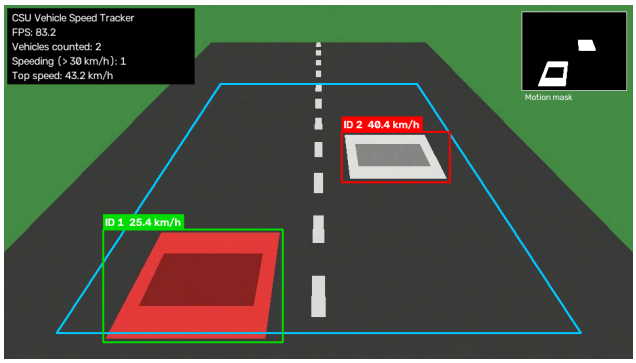


Fig. 1. Two vehicles tracked at once, each with its own ID and speed (25.4 and 40.4 km/h); the motion mask is inset at top right.

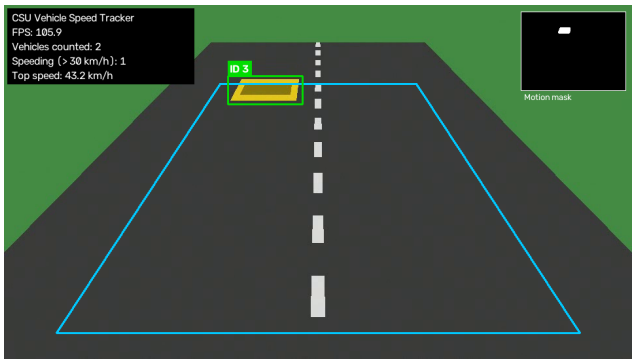


Fig. 2. A vehicle that has just entered the region: it is tracked (ID 3) but shows no speed until enough history is collected.

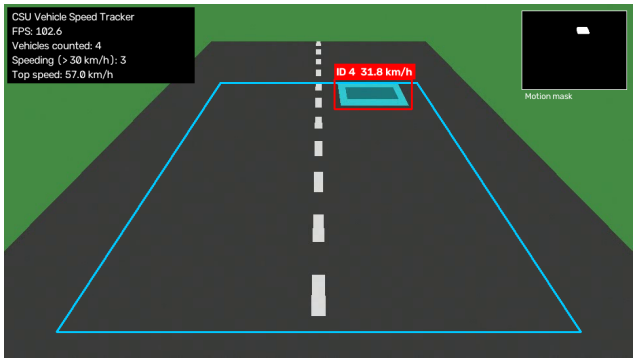


Fig. 3. A vehicle entering the calibrated region; the speed reading (31.8 km/h) is already available after a few frames.

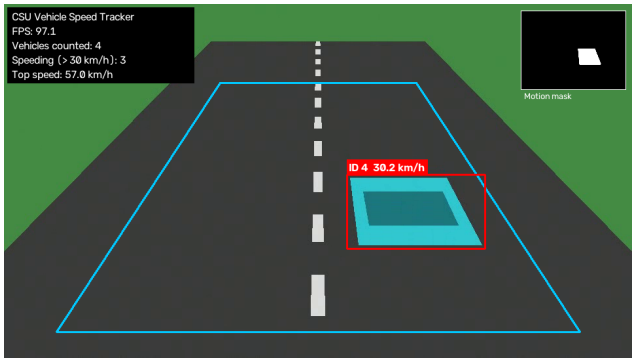


Fig. 4. A vehicle above the 30 km/h limit is drawn in red; the panel shows the count, speeders and top speed.

5. Accuracy, Behavior and Limitations

To measure accuracy with a known answer, the app was run on a synthetic 15-second traffic video generated by *make_sample_video.py*, in which four cars drive through a perspective-correct road scene at known speeds. All four vehicles were detected, tracked with a single ID each, and counted once:

Vehicle	True speed (km/h)	Measured (km/h)	Error
ID 1 (red)	25	24.8	-0.2
ID 2 (white)	40	40.4	+0.4
ID 3 (yellow)	55	55.8	+0.8
ID 4 (teal)	30	30.4	+0.4

The error was within 1 km/h (about 2% or less) on this test. The 30 km/h car was measured at 30.4 and therefore flagged as speeding, which shows that readings near the limit can fall on either side. These figures come from a controlled synthetic scene; results on the real campus camera will depend on calibration and should be tested and reported separately.

Limitations.

- The result depends on an accurate road calibration (measured width and length) and a fixed camera; moving the camera requires recalibration.
- Background subtraction runs on grayscale, so a vehicle whose brightness is close to the road is detected poorly. Shadows, rain, headlights and camera shake can create false blobs.
- Vehicles that overlap are merged into one blob, and the box center shifts slightly with perspective, which adds a small speed error.
- Speed is only measured inside the calibrated rectangle, and the tracker has no vehicle-type recognition, so pedestrians or bikes above the area threshold may be counted.

6. Conclusion

The project shows that classical OpenCV techniques are enough to build a working, low-cost speed monitor. Key learnings: the perspective transform is what turns pixels into real distances; segmentation quality (blur, thresholding, morphology) decides how stable the tracking is; and a median over several readings gives much steadier speeds than a single frame-to-frame value. Possible improvements: add a Haar cascade or a deep detector to ignore non-vehicles, use a Kalman filter for tracking through occlusions, log violations (time, ID, speed) to a CSV file, add night-time handling, and test with a radar gun or GPS-equipped vehicle to validate real-world accuracy.